

1. **Histórico raw de 48 meses desde trades**, con todos los campos originales y una marca `HISTORICAL / SEED / PRIOR_9M / RECENT_3M`.
2. **Variables creadas oficialmente**, tomadas de las tablas derivadas del proyecto. No las recalcularía para los meses antiguos porque eso cambiaría la definición del pipeline.

Esto mantiene el pull fiel al proyecto.

1. Seleccionar los dos casos

Primero creamos el positive y negative. Los dejo en la misma `observation_date` y con las principales features disponibles.

```
library(sparklyr)
library(dplyr)
```

```
sdf_sql(
  sc,
  "
  CREATE OR REPLACE TEMP VIEW fraud_case_pair AS
```

```
  WITH positive_case AS (
```

```
    SELECT
      party_id,
      observation_date,
      label_3m
```

```
  FROM bu_frd.model_features_ab_final_2022_2025
```

```
  WHERE label_3m = 1
    AND balance_change_available_flag = 1
    AND utilization_change_available_flag = 1
```

```
  ORDER BY
    observation_date,
    party_id
```

```
  LIMIT 1
),
```

```
negative_case AS (
```

```
  SELECT
    f.party_id,
    f.observation_date,
```

```

        f.label_3m

FROM bu_frd.model_features_ab_final_2022_2025 f

CROSS JOIN positive_case p

WHERE f.label_3m = 0
      AND f.observation_date = p.observation_date
      AND f.balance_change_available_flag = 1
      AND f.utilization_change_available_flag = 1

ORDER BY f.party_id

LIMIT 1
)

SELECT
  'POSITIVE' AS case_type,
  party_id,
  observation_date,
  label_3m
FROM positive_case

UNION ALL

SELECT
  'NEGATIVE' AS case_type,
  party_id,
  observation_date,
  label_3m
FROM negative_case
"
)

```

Revísalos:

```

case_pair <- sdf_sql(
  sc,
  "
  SELECT *
  FROM fraud_case_pair
  ORDER BY label_3m DESC
  "
) %>%
  collect()

case_pair

```

Debes obtener dos parties.

2. Crear calendario histórico de 48 meses

Este calendario es fundamental porque permitirá identificar claramente cuáles meses pertenecen al modelo y cuáles son únicamente historia anterior.

```
sdf_sql(
  sc,
  "
CREATE OR REPLACE TEMP VIEW fraud_case_calendar_48m AS

SELECT
  c.case_type,
  c.party_id,
  c.observation_date,
  c.label_3m,

  offset AS month_offset,

  last_day(
    add_months(
      c.observation_date,
      -offset
    )
  ) AS history_month,

  CASE
    WHEN offset BETWEEN 13 AND 47
      THEN 'HISTORICAL'

    WHEN offset = 12
      THEN 'SEED'

    WHEN offset BETWEEN 3 AND 11
      THEN 'PRIOR_9M'

    WHEN offset BETWEEN 0 AND 2
      THEN 'RECENT_3M'
  END AS period_type

FROM fraud_case_pair c
```

```
LATERAL VIEW explode(
  sequence(0, 47)
) s AS offset
"
)
```

Compruébalo:

```
case_calendar <- sdf_sql(
  sc,
  "
  SELECT *
  FROM fraud_case_calendar_48m

  ORDER BY
    case_type DESC,
    month_offset DESC
  "
) %>%
collect()
```

case_calendar

Por cada party deberías obtener 48 filas.

3. Pull raw completo de **trades**

Como quieres revisar el histórico, aquí sí podemos conservar **todas las columnas originales** mediante **t.***.

Eso evita perder cualquier campo de la fuente.

```
case_trades_history_48m <- sdf_sql(
  sc,
  "
  SELECT
    cal.case_type,
    cal.observation_date,
    cal.label_3m,

    cal.history_month,
    cal.month_offset,
    cal.period_type,
```

```

    t.*

FROM fraud_case_calendar_48m cal

LEFT JOIN trades t
  ON cal.party_id = t.party_id
  AND last_day(t.data_date) = cal.history_month

ORDER BY
  cal.case_type DESC,
  cal.month_offset DESC,
  t.acct_id
"
)

```

No haría `collect()` inmediatamente porque podría haber bastantes cuentas:

```

case_trades_history_48m %>%
  head(100) %>%
  collect()

```

Con esto vas a tener:

```

case_type
observation_date
label_3m
history_month
month_offset
period_type

party_id
trades_seq_num
acct_id
acct_num
acct_typ_cde
...
acct_mop_cde
cr_lmt_amt
curr_bal_amt
eff_dte
past_due_amt
...
data_date

```

más todas las demás columnas de `trades`.

4. Vista raw más manejable

Para revisar el caso probablemente no quieras ver las 80 columnas de `trades` simultáneamente.

Haría también una vista reducida con los campos más relevantes para el proyecto:

```
case_trades_history_view <- sdf_sql(  
  sc,  
  "  
  SELECT  
    cal.case_type,  
    cal.party_id,  
    cal.observation_date,  
    cal.label_3m,  
  
    cal.history_month,  
    cal.month_offset,  
    cal.period_type,  
  
    /* Account identification */  
    t.acct_id,  
    t.acct_num,  
    t.acct_typ_cde,  
    t.agcy_id_cde,  
  
    /* Account history */  
    t.open_dte,  
    t.clsd_dte,  
    t.end_dte,  
    t.frst_rpt_dte,  
    t.rem_frst_rpt_dte,  
  
    /* MOP / payment condition */  
    t.acct_mop_cde,  
    t.max_delq_rte_cde,  
    t.max_delq_amt,  
    t.max_delq_dte,  
    t.frst_delq_dte,  
  
    /* Exposure */  
    t.cr_lmt_amt,  
    t.curr_bal_amt,  
    t.high_cr_amt,  
  
    /* Past due / payment */  
    t.past_due_amt,
```

```

t.payt_amt,
t.payt_due_amt,
t.payt_dte,
t.payt_typ_cde,
t.payt_pttrn_txt,

/* Dates used for point-in-time control */
t.eff_dte,
t.data_date,

/* Other useful account descriptors */
t.acct_verf_cde,
t.acct_rte_prfl_cde,
t.ecoa_cde,
t.porf_typ_cde

FROM fraud_case_calendar_48m cal

LEFT JOIN trades t
  ON cal.party_id = t.party_id
  AND last_day(t.data_date) = cal.history_month

ORDER BY
  cal.case_type DESC,
  cal.month_offset DESC,
  t.acct_id
"
)

```

Luego:

```

case_trades_history_view %>%
  head(200) %>%
  collect()

```

Ojo con **cr_lmt_amt**

En tu lista aparece como:

```
cr_lmt_amt
```

Puede ser una **I** mayúscula o una **l** minúscula dependiendo de cómo se copió.

Como en nuestro desarrollo veníamos utilizando **cr_lmt_amt**, antes de ejecutar esa vista reducida valida:

```

sdf_sql(sc, "DESCRIBE trades") %>%
  filter(grepl("lmt|lmt|LMT|lmt", col_name)) %>%

```

collect()

Si el nombre real es diferente, cambiamos solo esa columna.

5. Resumen de cuántas cuentas aparecen por mes

Antes de mirar valores individuales, esta consulta ayuda a comprobar que el pull está funcionando.

```
case_history_month_summary <- sdf_sql(
  sc,
  "
  SELECT
    cal.case_type,
    cal.party_id,
    cal.observation_date,

    cal.history_month,
    cal.month_offset,
    cal.period_type,

    COUNT(t.acct_id) AS trade_rows,

    COUNT(DISTINCT t.acct_id)
      AS distinct_accounts,

    COUNT(DISTINCT t.acct_typ_cde)
      AS distinct_account_types

  FROM fraud_case_calendar_48m cal

  LEFT JOIN trades t
    ON cal.party_id = t.party_id
    AND last_day(t.data_date) = cal.history_month

  GROUP BY
    cal.case_type,
    cal.party_id,
    cal.observation_date,
    cal.history_month,
    cal.month_offset,
    cal.period_type
```

```
ORDER BY
  cal.case_type DESC,
  cal.month_offset DESC
"
) %>%
  collect()
```

case_history_month_summary

Esto debe darte exactamente:

$$48 \times 2 = 96$$

filas, aunque algunos meses antiguos podrían tener 0 cuentas si el party no estaba observable todavía.

Eso también es información válida del histórico.

6. Variables mensuales creadas oficialmente

Ahora agregamos las variables que nosotros ya construimos.

Aquí **no las recalculamos para HISTORICAL**.

Solo aparecen en los 12 meses oficiales para los que fueron construidas.

```
case_official_monthly_behavior <- sdf_sql(
  SC,
  "
  SELECT
    c.case_type,

    m.party_id,
    m.observation_date,
    m.label_3m,

    m.feature_month,
    m.month_offset,
    m.feature_window,

    /* Account counts */
```

```

m.reported_accounts,
m.active_accounts,
m.rated_active_accounts,
m.utilization_eligible_accounts,

/* Balance */
m.avg_balance_active,
m.total_balance_active,

/* Past due */
m.avg_past_due_active,
m.total_past_due_active,

/* Delinquency */
m.delinquent_active_accounts,
m.delinquent_account_share,

/* Utilization */
m.avg_utilization,

/* MOP behavior */
m.deterioration_count,
m.improvement_count,

m.adverse_entry_count,
m.adverse_resolved_count,

m.became_unrated_count,
m.left_unrated_count,

m.max_dpd_jump

FROM bu_frd.model_party_behavior_monthly_2022_2025 m

INNER JOIN fraud_case_pair c
  ON m.party_id = c.party_id
  AND m.observation_date = c.observation_date

ORDER BY
  c.case_type DESC,
  m.month_offset DESC
"
) %>%
collect()

case_official_monthly_behavior

```

Aquí esperamos:

$$2 \times 12 = 24$$

filas.

7. Weighted utilization mensual

Como weighted utilization quedó en su propia tabla:

```
case_official_monthly_utilization <- sdf_sql(  
  sc,  
  "  
  SELECT  
    c.case_type,  
    u.*  
  
  FROM bu_frd.model_party_utilization_monthly_2022_2025 u  
  
  INNER JOIN fraud_case_pair c  
    ON u.party_id = c.party_id  
    AND u.observation_date = c.observation_date  
  
  ORDER BY  
    c.case_type DESC,  
    u.month_offset DESC  
  "  
) %>%  
  collect()  
  
case_official_monthly_utilization
```

8. Ver también el SEED y las transiciones

La tabla mensual tiene 12 meses porque el SEED no es un feature month.

Pero para revisar MOP sí podemos mostrar los 13:

```
case_official_transitions_13m <- sdf_sql(  
  sc,  
  "  
  SELECT  
    c.case_type,
```

```

t.*

FROM bu_frd.model_trades_transition_13m_2022_2025 t

INNER JOIN fraud_case_pair c
  ON t.party_id = c.party_id
  AND t.observation_date = c.observation_date

ORDER BY
  c.case_type DESC,
  t.month_offset DESC,
  t.acct_id
"
)

```

Y revisar:

```

case_official_transitions_13m %>%
  head(200) %>%
  collect()

```

Aquí podrás ver:

$$M - 12 = SEED$$

hasta:

$$M0$$

con los flags derivados de las transiciones MOP.

9. Prior 9M y Recent 3M agregados

Este output permite ver cómo los doce meses se resumen antes de generar las features finales:

```

case_behavior_windows <- sdf_sql(
  sc,
  "
SELECT
  c.case_type,
  w.*

FROM bu_frd.model_party_behavior_windows_2022_2025 w

```

```

INNER JOIN fraud_case_pair c
  ON w.party_id = c.party_id
  AND w.observation_date = c.observation_date

ORDER BY c.case_type DESC
"
) %>%
collect()

case_behavior_windows

```

Aquí deberías encontrar las métricas correspondientes a:

*Prior*⁹

y:

*Recent*³

además de coverage y baseline SD.

10. Todas las engineered features finales

Y finalmente las dos filas que contienen las variables creadas que realmente pasarán al model development:

```

case_engineered_features <- sdf_sql(
  sc,
  "
SELECT
  c.case_type,

  f.party_id,
  f.observation_date,
  f.observation_year,
  f.label_3m,

  /* MOP behavior */
  f.deterioration_acceleration,
  f.adverse_acceleration,
  f.max_dpd_jump_recent_3m,

```

```

/* Active accounts */
f.active_accounts_change_vs_baseline,
f.active_accounts_changed_flag,

/* Balance */
f.balance_change_model,
f.log1p_abs_balance_change_model,
f.balance_change_available_flag,
f.balance_recent_months_available,
f.balance_prior_months_available,

/* Past due */
f.past_due_change_model,
f.past_due_change_available_flag,
f.past_due_recent_months_available,
f.past_due_prior_months_available,

/* Delinquency */
f.delinquency_share_change_model,
f.delinquency_change_available_flag,
f.delinquency_recent_months_available,
f.delinquency_prior_months_available,

/* Current utilization */
f.avg_utilization_change_model,
f.weighted_utilization_change_model,
f.utilization_change_available_flag,
f.utilization_recent_months_available,
f.utilization_prior_months_available,

/* Selected Z challengers */
f.active_accounts_z_model,
f.active_accounts_z_available_flag,

f.avg_utilization_z_model,
f.avg_utilization_z_available_flag,

f.weighted_utilization_z_model,
f.weighted_utilization_z_available_flag

FROM bu_frd.model_features_ab_final_2022_2025 f

INNER JOIN fraud_case_pair c
  ON f.party_id = c.party_id
  AND f.observation_date = c.observation_date

ORDER BY f.label_3m DESC
"

```

```
) %>%
  collect()

case_engineered_features
```

Este output debería tener exactamente:

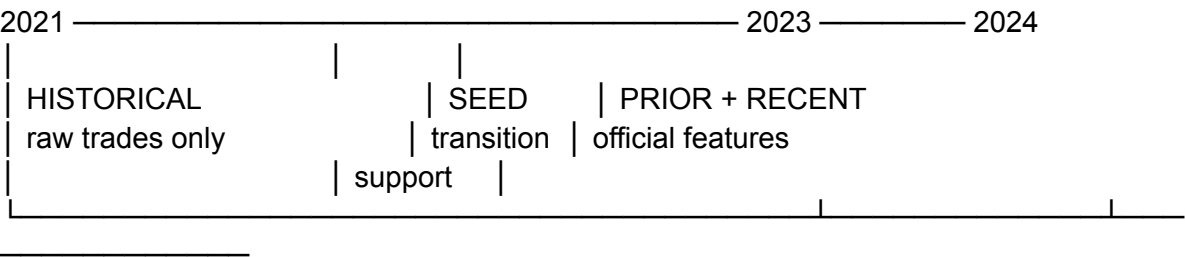
2 rows

Qué tendrás finalmente

Yo mantendría estos outputs porque cada uno tiene una función distinta:

Output	Granularidad	Qué muestra
case_history_month_summary	party-month	48 meses y profundidad del historial
case_trades_history_48m	party-account-month	raw histórico completo
case_official_monthly_behavior	party-month	las variables mensuales creadas en los 12 meses oficiales
case_official_monthly_utilization	party-month	avg/weighted utilization oficial
case_official_transitions_3m	party-account-month	SEED + MOP transitions
case_behavior_windows	party	Prior9 / Recent3 aggregates
case_engineered_features	party	todas las features finales

Visualmente, para un caso cuyo `observation_date` fuera diciembre de 2024, tendríamos:



Así obtienes exactamente lo que querías: **puedes ver los meses anteriores libremente**, pero sigue quedando completamente claro qué período generó las features oficiales del proyecto.